

PROJECT : PANZER

CLASSIFIED

Quentin Hordies et Horus Plancq Marchal

AU1 – Q2 ,2026

Sommaire

• Introduction : Contexte et objectifs.	2
• Cahier des charges	3
• Rappels théoriques : Rappel du fonctionnement théorique du capteur utilisé	4
• SR-04	4
• BMP280	5
• Schéma de montage :.....	6
• Schéma block.....	6
• Expliquer le câblage.....	7
LED :.....	7
SR04 :	7
Bmp280 :	7
Alimentation :.....	7
• Choix motivé et dimensionnement des composant : Justification	8
• Partie Logicielle :.....	9
• Algorigramme.....	9
• Table des I/O	10
Explication du code.....	11
Bmp280.py	11
capteurs.py.....	13
main.py	14
• Prototype : Décrire la réalisation physique de votre projet	18
Le châssis :	18
L'électronique :	20
Les chenilles :.....	20
• Conclusion : Analyse critique.....	22

• Introduction : Contexte et objectifs.

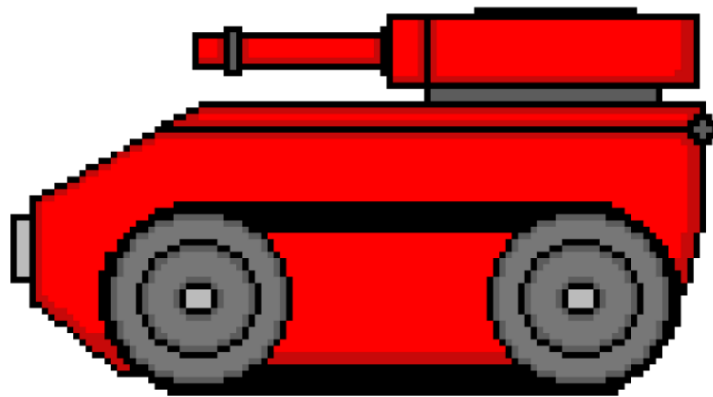
Le projet Panzer, inspiré des UGV, est un mini char autonome qui se repère dans son environnement et se déplace de lui-même, il récolte certaines informations et après les avoir analysés transmet un signal lumineux soit positif, soit négatif a ceux qui peuvent l'apercevoir.

L'objectif de panzer est donc de servir d'éclaireur, ses chenilles lui permettent d'aller sur certains terrains difficilement praticables et de prendre de l'avance sur des humains afin de leur donné des informations sur l'atmosphère dans l'environnement dans lequel ils évolueront.



• Cahier des charges

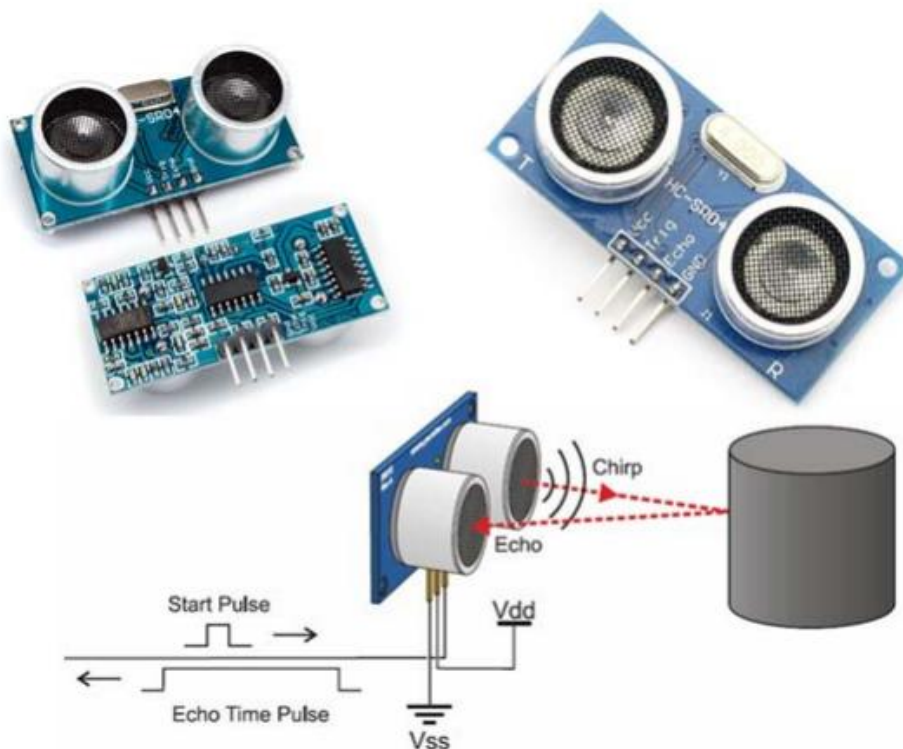
Le projet consiste en 1 mini tank qui se dirige de manière autonome et évite les collisions grâce à un capteur de distance ultrasons (SR04), il embarque un capteur barométrique (BMP280) qui va indiquer grâce à 2 LED (un verte et un rouge) si les conditions atmosphériques dans lesquels il se trouvent sont idéal pour l'homme ou non, un peu comme un drone de reconnaissance qu'on enverrait en éclaireur. Sa vitesse est progressive en fonction de la distance des obstacles en face de lui, au plus il s'approche au plus il ralenti, et si l'obstacle est trop proche il change de direction pour l'éviter



• Rappels théoriques : Rappel du fonctionnement théorique du capteur utilisé

•SR-04

Le SR04 est un télémètre à ultrasons, c'est-à-dire qu'il utilise des ultrasons pour calculer une distance. L'un des 2 cylindre à l'avant du capteur est un émetteur, l'autre un récepteur, lorsque l'émetteur envoie un signal (Trig) le receveur attend que l'ultrason heurte une surface et reviennent vers lui (Echo) il mesure le temps entre l'envoi et la réception, le divise en 2 puis on peut en déduire une distance.



Electrical Parameters	Value
Operating Voltage	3.3Vdc ~ 5Vdc
Quiescent Current	<2mA
Operating Current	15mA
Operating Frequency	40KHz
Operating Range & Accuracy	2cm ~ 400cm (1in ~ 13ft) $\pm$ 3mm
Sensitivity	-65dB min
Sound Pressure	112dB
Effective Angle	15°
Connector	4-pins header with 2.54mm pitch
Dimension	45mm x 20mm x 15mm
Weight	9g

• BMP280

Le BMP280 est un capteur conçu par Bosch, c'est un capteur de pression atmosphérique et de température.

J'utilise un module qui se base sur ce capteur conçu par Adafruit .

Il est conçu pour offrir une faible consommation, une bonne stabilité à long terme, et une grande précision dans une large gamme de température et de pression.



Datasheet :

Plage de pression : 300 à 1100 hPa (équivalent de 0 à ~9000 m d'altitude)

Température : -40 °C à +85 °C

Précision pression : ± 1 hPa ($\sim \pm 8$ m d'altitude)

Précision température : ± 1.0 °C

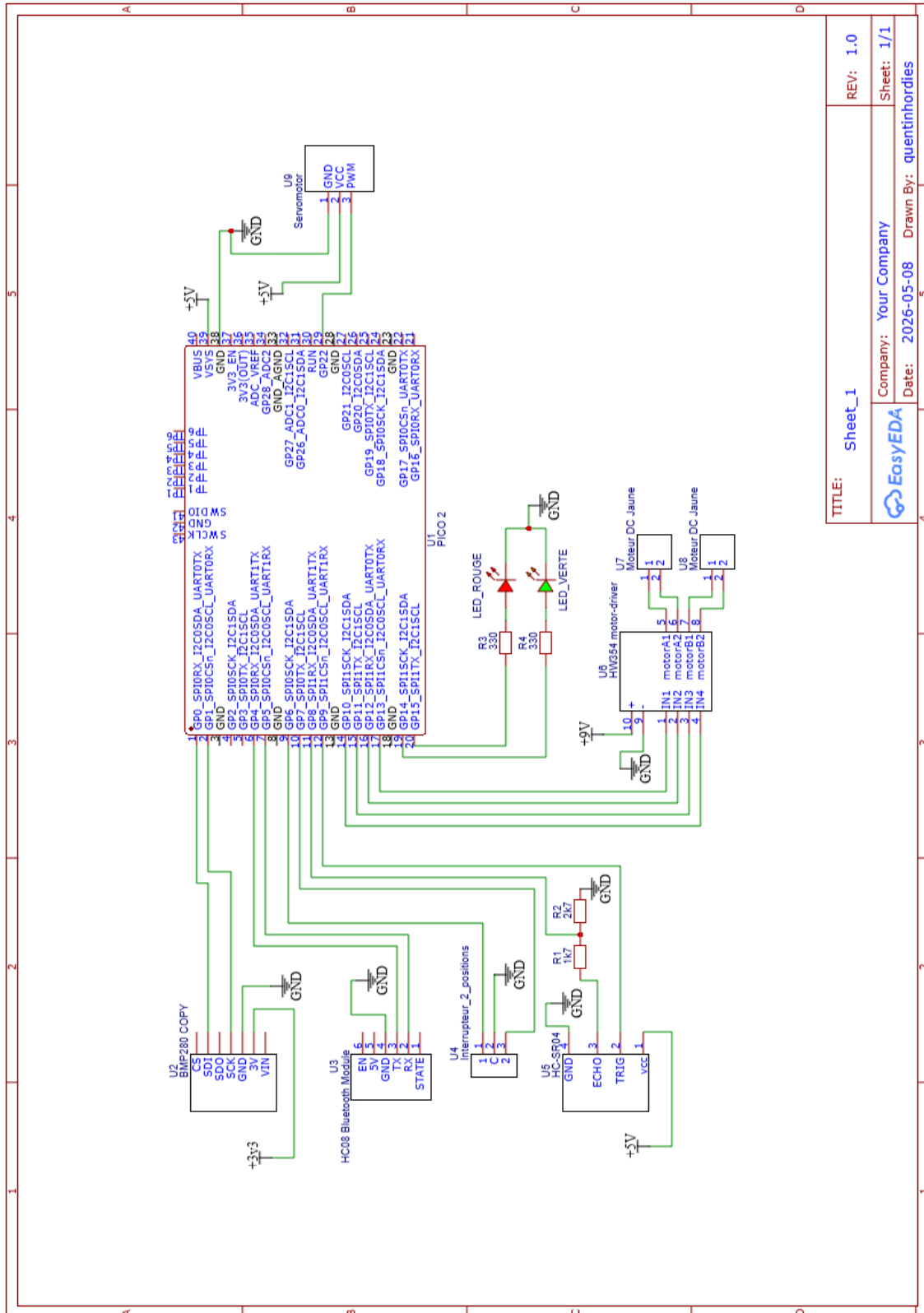
Interface : I2C (adresse 0x76 ou 0x77)

Taux d'échantillonnage : jusqu'à 157 Hz

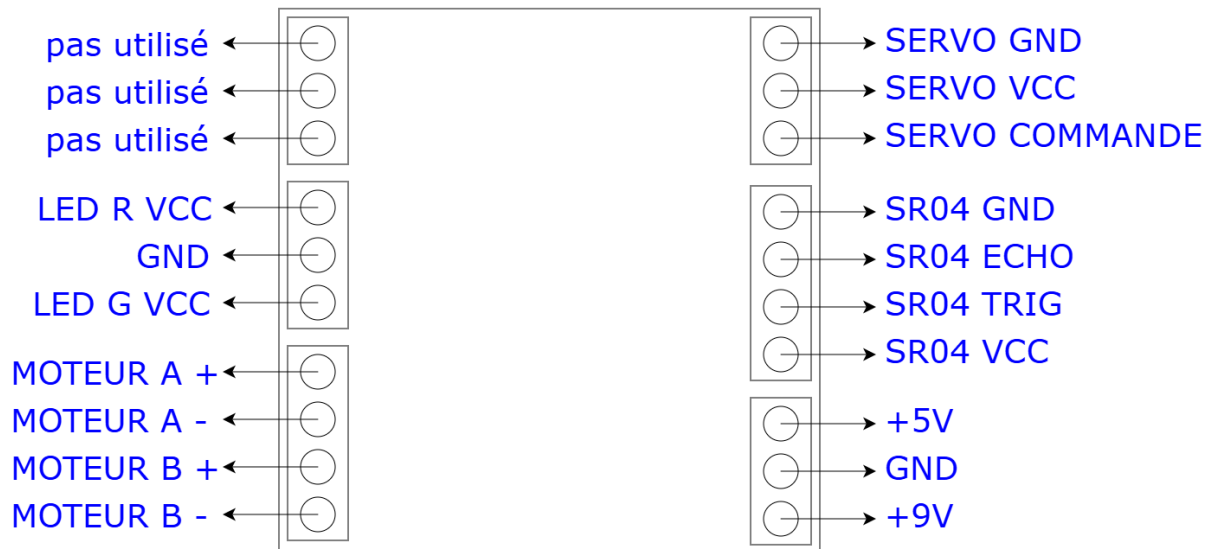
Consommation : type. 2.7 μ A en mode normal, <1 μ A en veille

Alimenter en 3V3

- Schéma de montage :
- Schéma block



TITLE: Sheet_1	REV: 1.0
Company: Your Company	Sheet: 1/1
Date: 2026-05-08	Drawn By: quentinhordies



Câblage des borniers sur la PCB

- Expliquer le câblage

LED :

Les LEDs sont simplement branchés à des GPIO et à la masse, en passant par une résistance de 330 ohms pour diviser la tension, elles servent d'indicateurs lumineux activés par le pi pico.

SR04 :

Le SR04 a son propre mode fonctionnement (Echo, Trig), la seul spécifié est l'alimentation, le capteur est fait pour être alimenté en 5V, ce qui est fait ici, il faut donc dans ce cas diviser la tension qui rentre dans le Raspberry grâce à un pont diviseur de tension pour ne pas risquer de cramer le microcontrôleur.

Bmp280 :

Le bmp280 est un capteur simple branché en I2c grâce à un câble Signal et un Data, il est alimenté en 3V3 par le Raspberry, son adresse est 0x77.

Alimentation :

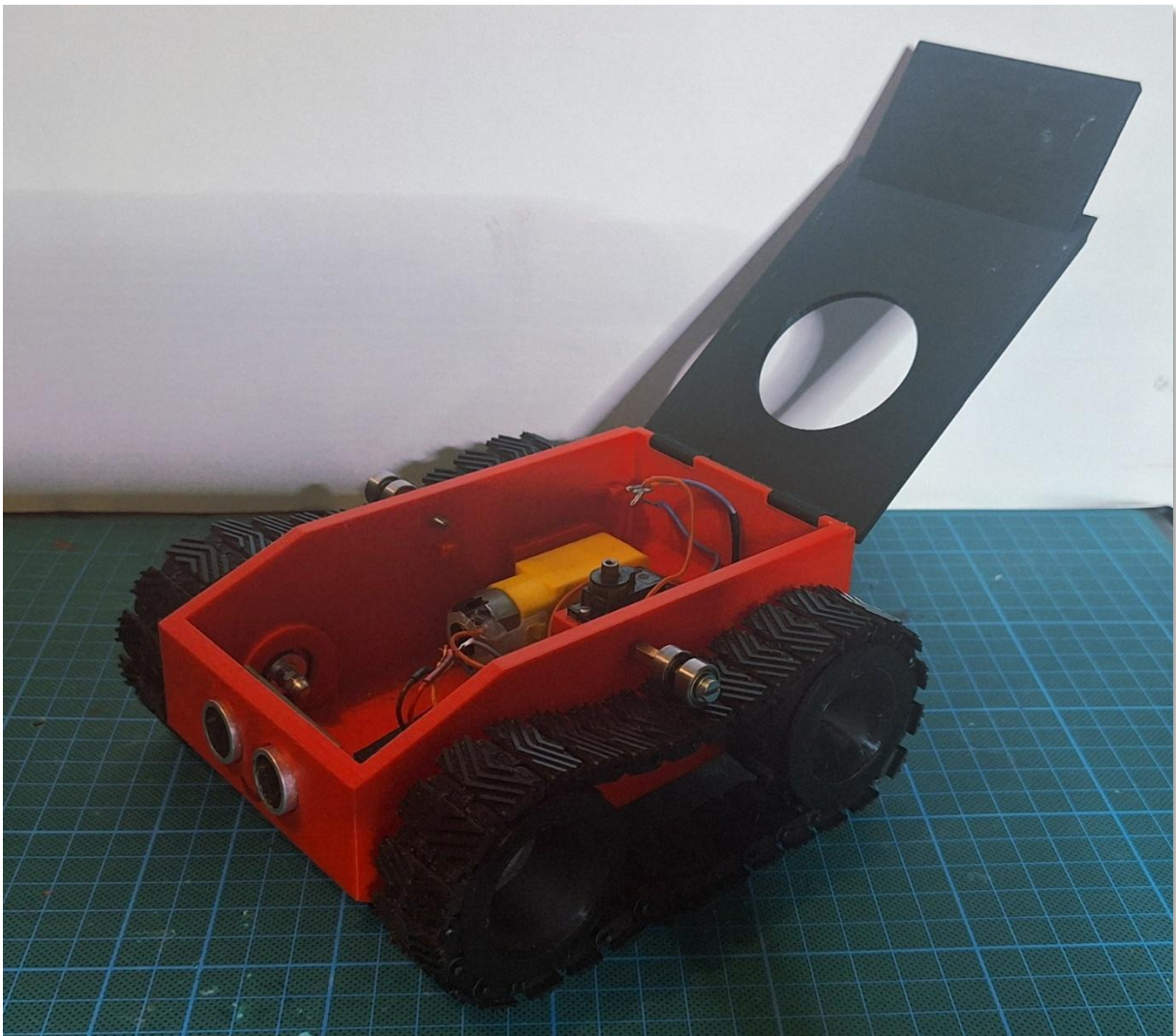
L'alimentation est divisée en 3 : premièrement, une batterie 9V alimente le driver moteur qui permet au moteur d'avancer. Ensuite une batterie 5V viens alimenter le servomoteur, le SR04, et la Raspberry pi pico. Pour finir le BMP280 est alimenté en 3V3 directement depuis le microcontrôleur

- **Choix motivé et dimensionnement des composant : Justification**

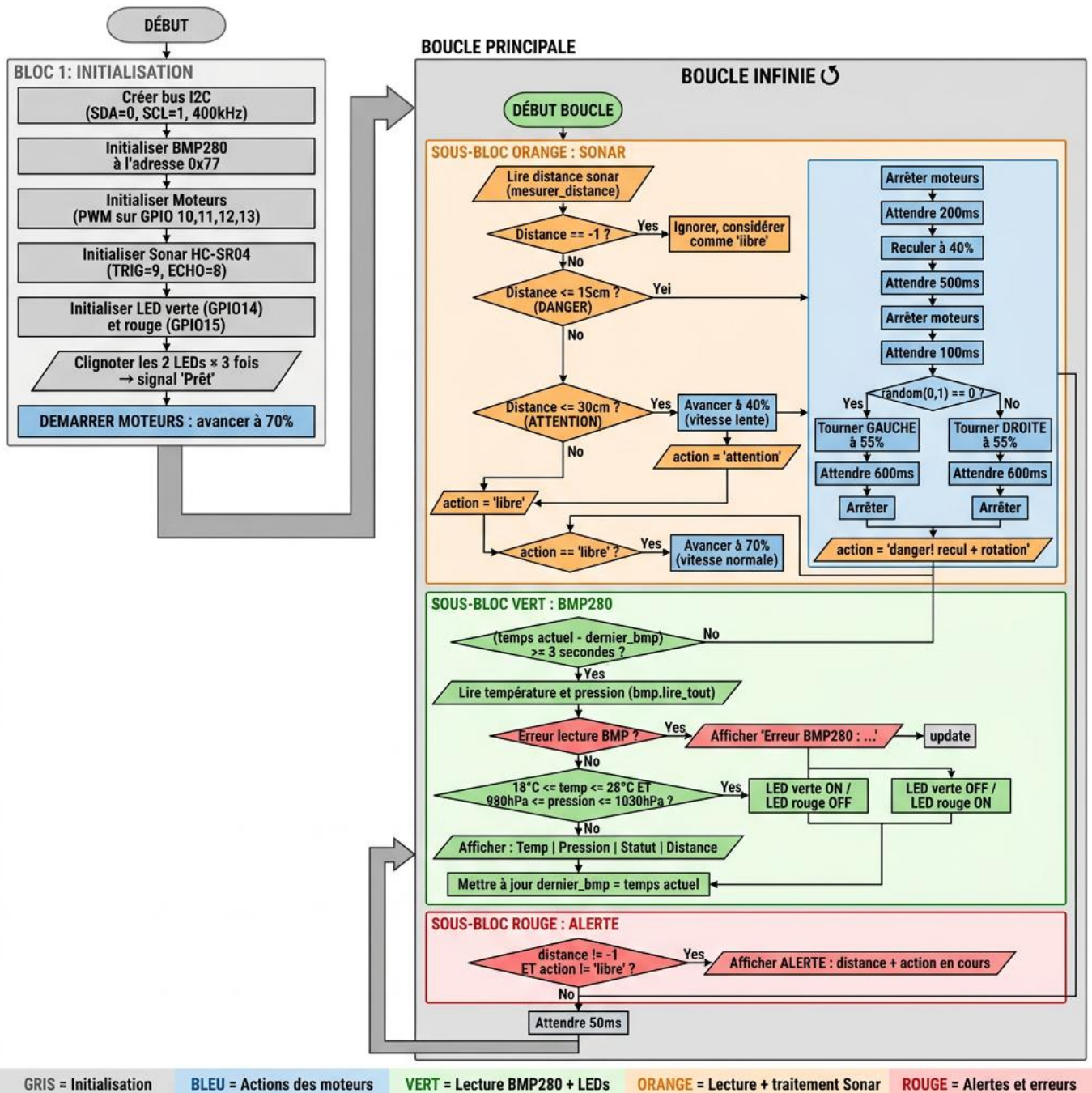
J'ai choisi des capteurs avec lesquels j'ai déjà eu l'occasion de travailler et que j'avais chez moi afin de réduire un maximum les couts du projet

La taille a été un vrai défi puisque panzer est complètement autonome, il fallait trouver la bonne taille entre suffisamment grand pour pouvoir tout rentré et suffisamment petit pour que les moteurs dc ne soient pas surchargés, c'est pour cela que plusieurs versions et taille du tank existe.

C'est dans cette optique d'économie de place que le tank n'est équipé que d'un seul capteur de distance et non de deux (ce qui permettrait de mieux éviter certains types d'obstacles).



- Partie Logicielle :
- Algorithme



• Table des I/O

N°GPIO	Nom dans le code	Direction	Type de signal	Composant connecté	Rôle
GPIO 0	I2C SDA	Bidirectionnel	I2C Data	BMP280	Ligne de données I2C
GPIO 1	I2C SCL	Sortie	I2C Clock	BMP280	Horloge I2C (400kHz)
GPIO 13	PIN IN1	Entrée	Numérique	HC-SR04	Réception écho ultrason
GPIO 12	PIN IN2	Sortie	Numérique impulsion	HC-SR04	Déclenchement mesure sonar
GPIO 11	PIN IN3	Sortie	PWM 1kHz	MX1508 / HW-354	Moteur droit - sens 2
GPIO 10	PIN IN4	Sortie	PWM 1kHz	MX1508 / HW-354	Moteur droit - sens 1
GPIO 9	PIN TRIG	Sortie	PWM 1kHz	MX1508 / HW-354	Moteur gauche - sens 2
GPIO 8	PIN ECHO	Sortie	PWM 1kHz	MX1508 / HW-354	Moteur gauche - sens 1
GPIO 14	PIN LED VERTE	Sortie	Numérique ON/OFF	LED Verte	Environnement normal
GPIO 15	PIN LED ROUGE	Sortie	Numérique ON/OFF	LED Rouge	Environnement anormal

Explication du code

Bmp280.py

La classe et l'adresse I2C

Création de la classe BMP280, nous utilisons l'adresse I2C 0x76 pour ce capteur. Nous vérifions si le capteur est bien connecté.

```
class BMP280:
    I2C_ADDRESS = 0x76
    def __init__(self, i2c, address=0x76):
        self.i2c = i2c
        self.address = address
        devices = self.i2c.scan()
        if self.address not in devices:
            print(f"BMP280 non trouvé! Appareils détectés: {devices}")
        else: print("BMP280 trouvé!")
```

Initialisation hardware

Lecture des données d'étalonnage du capteur, début de la mesure, réglage de la vitesse d'échantillonnage.

```
self._lire_calibration()
self.i2c.writeto_mem(self.address, 0xF4, bytes([0x27]))
self.i2c.writeto_mem(self.address, 0xF5, bytes([0xA0]))
```

Lecture de calibration

Lecture des 24 octets de mémoire du capteur sur base de l'adresse 0x88 afin de récupérer les valeurs de correction propres au capteur.

```
def _lire_calibration(self):
    cal = self.i2c.readfrom_mem(self.address, 0x88, 24)
    self.dig_T1 = (cal[1] << 8) | cal[0]
    self.dig_T2 = self._signe((cal[3] << 8) | cal[2])
    self.dig_T3 = self._signe((cal[5] << 8) | cal[4])
    self.dig_P1 = (cal[7] << 8) | cal[6]
    self.dig_P2 = self._signe((cal[9] << 8) | cal[8])
    self.dig_P3 = self._signe((cal[11] << 8) | cal[10])
    self.dig_P4 = self._signe((cal[13] << 8) | cal[12])
    self.dig_P5 = self._signe((cal[15] << 8) | cal[14])
    self.dig_P6 = self._signe((cal[17] << 8) | cal[16])
    self.dig_P7 = self._signe((cal[19] << 8) | cal[18])
    self.dig_P8 = self._signe((cal[21] << 8) | cal[20])
    self.dig_P9 = self._signe((cal[23] << 8) | cal[22])
```

Fonction signe

Rectification des valeurs de calibration.

```
def _signe(self, valeur):
    if valeur > 32767:
        valeur -= 65536
    return valeur
```

Lecture des données brutes

Lecture des données brutes du capteur dans les 6 octets depuis 0xF7, les 3 premiers sont la pression, les 3 suivants la température.

```
def _lire_brut(self):
    data = self.i2c.readfrom_mem(self.address, 0xF7, 6)
    pression_brute = ((data[0] << 16) | (data[1] << 8) | data[2]) >> 4
    temp_brute = ((data[3] << 16) | (data[4] << 8) | data[5]) >> 4
    return pression_brute, temp_brute
```

Calcul de la température

Application de la formule officielle pour passer de la valeur brute au °C.

```
def lire_temperature(self):
    _, temp_brute = self._lire_brut()
    var1 = (temp_brute / 16384.0 - self.dig_T1 / 1024.0) * self.dig_T2
    var2 = (temp_brute / 131072.0 - self.dig_T1 / 8192.0) ** 2 * self.dig_T3
    self.t_fine = var1 + var2
    temperature = self.t_fine / 5120.0
    return round(temperature, 2)
```

Calcul de la pression

Application de la formule officielle pour passer de la valeur brute à l'hectopascal (hPa), en prenant en compte la température.

```
def lire_pression(self):
    pression_brute, _ = self._lire_brut()
    self.lire_temperature()
    var1 = self.t_fine / 2.0 - 64000.0
    var2 = var1 * var1 * self.dig_P6 / 32768.0
    var2 = var2 + var1 * self.dig_P5 * 2.0
    var2 = var2 / 4.0 + self.dig_P4 * 65536.0
    var1 = (self.dig_P3 * var1 * var1 / 524288.0 + self.dig_P2 * var1) / 524288.0
    var1 = (1.0 + var1 / 32768.0) * self.dig_P1
    if var1 == 0:
        return 0
    pression = 1048576.0 - pression_brute
    pression = ((pression - var2 / 4096.0) * 6250.0) / var1
    var1 = self.dig_P9 * pression * pression / 2147483648.0
    var2 = pression * self.dig_P8 / 32768.0
    pression = pression + (var1 + var2 + self.dig_P7) / 16.0
    return round(pression / 100.0, 2)
```

capteurs.py

Initialisation du sonar

Initialisation des pin "trig" et "echo" pour le déclenchement et la réception du signal, mise à zéro de "trig".

```
class HCSR04:
    def __init__(self, trig_pin, echo_pin):
        self.trig = machine.Pin(trig_pin, machine.Pin.OUT)
        self.echo = machine.Pin(echo_pin, machine.Pin.IN)
        self.trig.value(0)
        time.sleep_ms(100)
```

Mesure de distance

Calcule la distance entre le capteur et un obstacle, selon le temps que mettent les ultrasons à revenir. Si le signal ne revient pas, c'est qu'il n'y a pas d'obstacle.

```
def mesurer_distance(self):
    self.trig.value(0)
    time.sleep_us(2)
    self.trig.value(1)
    time.sleep_us(10)
    self.trig.value(0)
    timeout = time.ticks_us()
    while self.echo.value() == 0:
        if time.ticks_diff(time.ticks_us(), timeout) > 30000:
            return -1
    debut = time.ticks_us()
    while self.echo.value() == 1:
        if time.ticks_diff(time.ticks_us(), debut) > 30000:
            return -1
    fin = time.ticks_us()
    duree = time.ticks_diff(fin, debut)
    distance = duree / 58.0
    return round(distance, 1)
```

Mesure moyenne

Fait la moyenne de 3 mesures pour une plus grande fiabilité.

```
def mesurer_moyenne(self, nb_mesures=3):
    mesures = []
    for _ in range(nb_mesures):
        dist = self.mesurer_distance()
        if dist != -1 and dist <= 400:
            mesures.append(dist)
        time.sleep_ms(20)
    if mesures:
        return round(sum(mesures) / len(mesures), 1)
    else:
        return -1
```

main.py

Classe Moteurs

Initialisation des moteurs et de leur fréquence, définition du PWM qui permet la modulation de vitesse selon l'envoi de courant à une certaine fréquence.

```
class Moteurs:
    VITESSE_MAX = 65535
    def __init__(self, in1, in2, in3, in4):
        self.in1 = machine.PWM(machine.Pin(in1))
        self.in2 = machine.PWM(machine.Pin(in2))
        self.in3 = machine.PWM(machine.Pin(in3))
        self.in4 = machine.PWM(machine.Pin(in4))
        self.in1.freq(1000)
        self.in2.freq(1000)
        self.in3.freq(1000)
        self.in4.freq(1000)
```

Conversion vitesse

Conversion de la vitesse du pourcentage (0 - 100) vers la grandeur PWM (0 - 65535) avec protection empêchant le dépassement des valeurs.

```
def _vitesse_pwm(self, pourcentage):
    pourcentage = max(0, min(100, pourcentage))
    return int(pourcentage / 100.0 * self.VITESSE_MAX)
```

Commandes des moteurs

Fait aller les moteurs en avant ou en arrière en fonction de la direction souhaiter.

```
def avancer(self, vitesse=70):
    pwm = self._vitesse_pwm(vitesse)
    self.in1.duty_u16(pwm)
    self.in2.duty_u16(0)
    self.in3.duty_u16(pwm)
    self.in4.duty_u16(0)
def tourner_gauche(self, vitesse=60):
    pwm = self._vitesse_pwm(vitesse)
    self.in1.duty_u16(0)
    self.in2.duty_u16(pwm)
    self.in3.duty_u16(pwm)
    self.in4.duty_u16(0)
```

Configuration des pins

Définition des GPIO connecter pour meilleur lisibilité du Code.

```
I2C_SDA = 0
I2C_SCL = 1
PIN_IN1 = 13
PIN_IN2 = 12
PIN_IN3 = 11
PIN_IN4 = 10
PIN_TRIG = 9
PIN_ECHO = 8
PIN_LED_VERTE = 14
PIN_LED_ROUGE = 15
```

Seuils et paramètres

Définition des Variable, seuils et paramètre.

```
DISTANCE_DANGER      = 15
DISTANCE_ATTENTION   = 30

VITESSE_NORMALE      = 70
VITESSE_LENTE        = 40
VITESSE_ROTATION     = 55

TEMP_MIN_OK          = 18.0
TEMP_MAX_OK          = 28.0
PRESSION_MIN_OK      = 980.0
PRESSION_MAX_OK      = 1030.0

INTERVALLE_BMP       = 3.0
```

Initialisation des composants

Création de bus I2C a 400kHz et initialisation des composant avec les différente class.

```
i2c = machine.I2C(0, sda=machine.Pin(I2C_SDA), scl=machine.Pin(I2C_SCL), freq=400000)
bmp = BMP280(i2c, address=119)
moteurs = Moteurs(PIN_IN1, PIN_IN2, PIN_IN3, PIN_IN4)
sonar = HCSR04(PIN_TRIG, PIN_ECHO)
led_verte = machine.Pin(PIN_LED_VERTE, machine.Pin.OUT)
led_rouge = machine.Pin(PIN_LED_ROUGE, machine.Pin.OUT)
```

Séquence de démarrage

Fait clignoter les LEDs 3 fois.

```
for _ in range(3):
    led_verte.value(1)
    led_rouge.value(1)
    time.sleep_ms(300)
    led_verte.value(0)
    led_rouge.value(0)
    time.sleep_ms(300)
```

Fonction évaluation environnement

Evalue l'environnement en fonction de la température et pression détecter par le capteur et des Seuils définie.

```
def evaluer_environnement(temperature, pression):
    temp_ok = TEMP_MIN_OK <= temperature <= TEMP_MAX_OK
    pression_ok = PRESSION_MIN_OK <= pression <= PRESSION_MAX_OK
    return temp_ok and pression_ok
```

Fonction LEDs

Actualisation des LEDs en fonction de l'évaluation de l'environnement.

```
def mettre_a_jour_leds(environnement_ok):
    if environnement_ok:
        led_verte.value(1)
        led_rouge.value(0)
    else:
        led_verte.value(0)
        led_rouge.value(1)
```

Fonction évitement d'obstacles

Modification de la vitesse en fonction de la distance établie par le capteur et change de direction si l'obstacle est trop proche.

```
def eviter_obstacle(distance):  
    if distance == -1:  
        return "libre"  
    elif distance <= DISTANCE_DANGER:  
        moteurs.arreter()  
        time.sleep_ms(200)  
        moteurs.reculer(VITESSE_LENTE)  
        time.sleep_ms(500)  
        moteurs.arreter()  
        time.sleep_ms(100)  
        if random.randint(0, 1) == 0:  
            moteurs.tourner_gauche(VITESSE_ROTATION)  
            direction = "gauche"  
        else:  
            moteurs.tourner_droite(VITESSE_ROTATION)  
            direction = "droite"  
        time.sleep_ms(600)  
        moteurs.arreter()  
        return f"danger! recul + rotation {direction}"  
    elif distance <= DISTANCE_ATTENTION:  
        moteurs.avancer(VITESSE_LENTE)  
        return f"attention ({distance}cm) - ralenti"  
    else:  
        return "libre"
```

Boucle principale

Applique en boucle les différentes fonctions pour avoir un robot complètement autonome.

```
dernier_bmp = time.time()
environnement_ok = True
moteurs.avancer(VITESSE_NORMALE)

while True:
    distance = sonar.mesurer_distance()
    action = eviter_obstacle(distance)
    if action == "libre":
        moteurs.avancer(VITESSE_NORMALE)
    maintenant = time.time()
    if maintenant - dernier_bmp >= INTERVALLE_BMP:
        try:
            temperature, pression = bmp.lire_tout()
            environnement_ok = evaluer_environnement(temperature, pression)
            mettre_a_jour_leds(environnement_ok)
            statut = "OK" if environnement_ok else "ANORMAL"
            txt_distance = f"{distance}cm" if distance != -1 else "Erreur
capteur"
            print(
                f"Temp: {temperature}°C | "
                f"Pression: {pression}hPa | "
                f"Environnement: {statut} | "
                f"Distance: {txt_distance}"
            )
        except Exception as e:
            print(f"Erreur BMP280: {e}")
            dernier_bmp = maintenant
    if distance != -1 and action != "libre":
        print(f"[ALERTE] Distance active: {distance}cm | Action: {action}")
    time.sleep_ms(50)
```

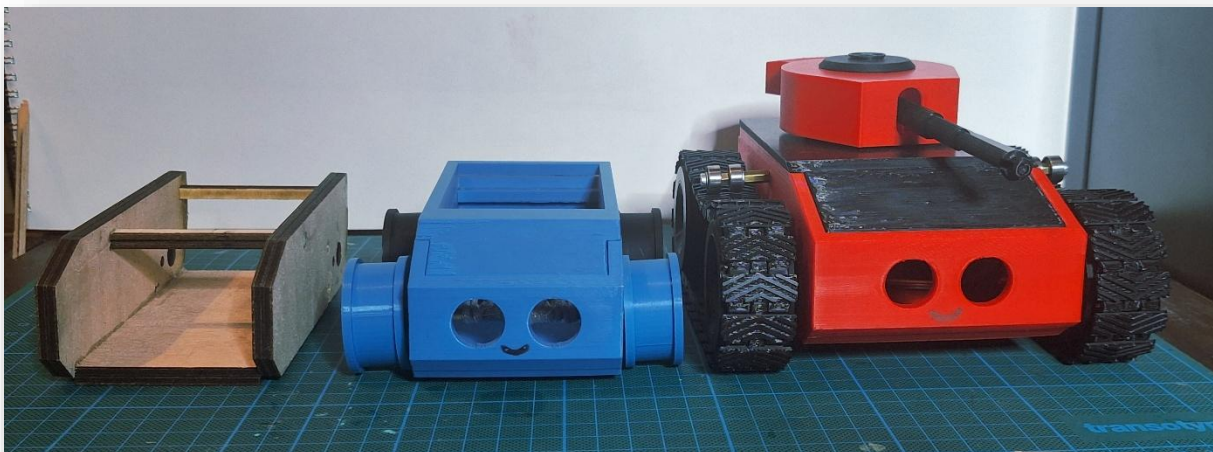
- **Prototype : Décrire la réalisation physique de votre projet**

Panzer est avant tout un châssis imprimé en 1 pièce en PLA au centre se trouve un logement pour le servomoteur, au 4 coins on y voit des roulements à bille qui peuvent être visé en place grâce au 4 vis sous le châssis du tank

A l'avant 2 trou permettent d'insérer l'émetteur et le récepteur du télémètre ultrasons

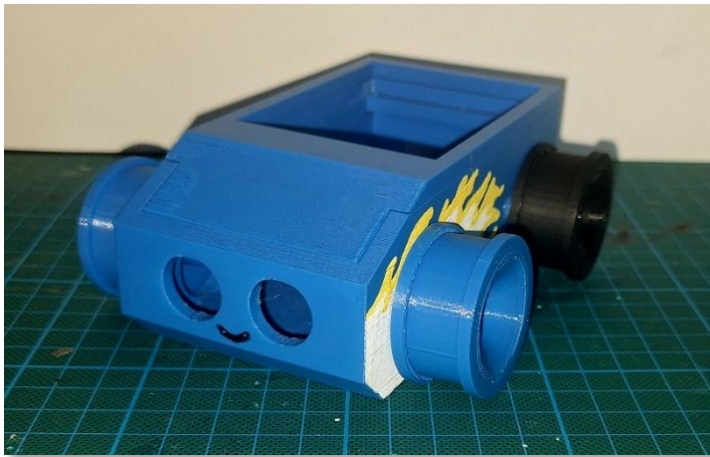
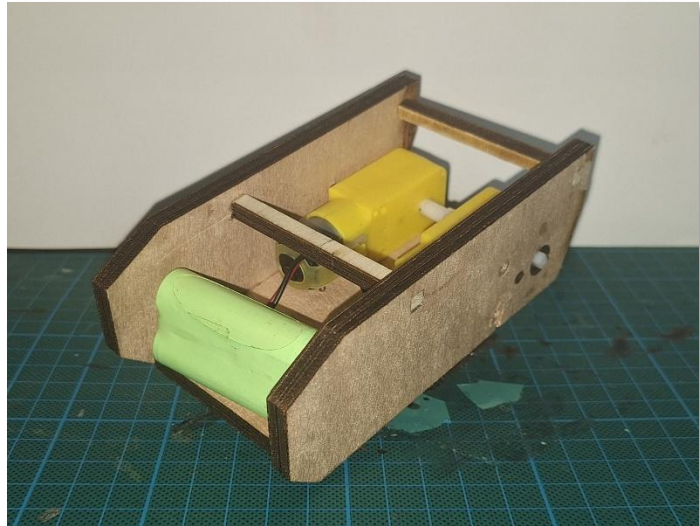
Enfin on retrouve une chenille de chaque côté qui, entraîné individuellement par un moteur DC, permettent à Panzer de se déplacer.

Le châssis :



(Dans l'ordre de création de gauche à droite)

Il y a eu 3 itérations du châssis, la première surnommé « carcasse » est un modèle en bois découpé au laser et collé, il a permis de faire un test d'encombrement pour la version suivante.



« Robi » est la 2ème itération, il a eu droit à sa propre PCB et est la première maquette « fonctionnelle » le problème de « Robi » est le manque de place, que ce soit pour la batterie ou le servomoteur.

« Panzer » est la version finale (pour l'instant) il est plus gros et a des chenilles plus large et épaisse, là où « Robi » n'avait que 2 roulements à l'avant pour les roues libre des chenilles panzer passe à 4 roulements, 2 ont été ajoutés afin de supporter le travail des moteurs et la contrainte imposée par les chenilles, il est plus large afin d'accueillir le servomoteur et la batterie. Il est équipé d'une PCB sur mesure ainsi que d'un capot supérieur sur charnière pour le refermer.

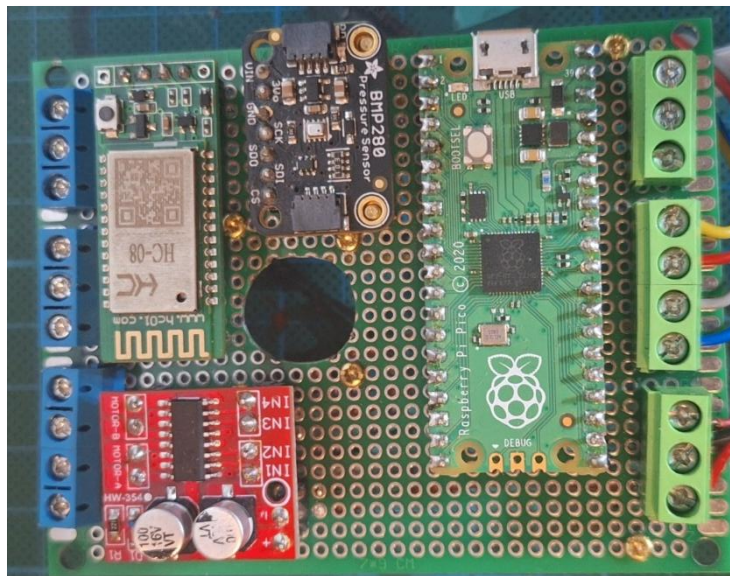


L'électronique :

La toute première version était une simple breadboard posé sur le dessus du châssis numéro 1.

Ensuite nous sommes très vite passé à une PCB très simple (et moche) qui s'articulait en 2 parties attachés au Raspberry

Pour finir on a créé une PCB maison grâce à des fils et une carte percée ce qui a permis d'avoir une base solide et facile à déplacer pour la version finale



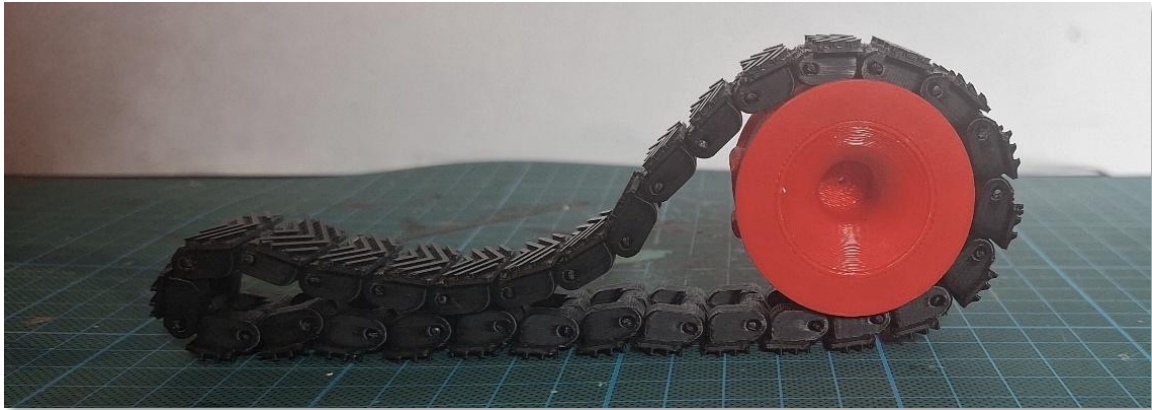
Les chenilles :

C'est de loin la partie qui a nécessité le plus de travail, au début nous somme parti sur des couronnes en TPU (filament souple) mais elles avaient tendance à glisser si elles n'étaient pas assez tendues et elle opposait une grosse résistance aux moteurs.



La version suivante était une chaîne relativement fine, plus précise mais bcp plus fragile donc pas adapté, elle pouvait casser simplement en déraillant de la roue motrice.

Pour la dernière version, nous sommes parti sur une chaîne avec des maillons individuels reliés par un filament de bobine ; ces maillons sont beaucoup plus épais et résistants, il aura quand même fallu plusieurs itérations pour trouver les bons écarts pour permettre à la chaîne de bouger librement sans trop de jeu.



Pour les accompagner, une série de roues dentées ont été faites jusqu'à trouver le « sweet spot ».



• Conclusion : Analyse critique

- 1- Le système de batterie : actuellement le système de batterie en 5V et 9V séparé n'est pas optimale ne serait-ce que pour le rechargement de plus elles sont simplement déposées (en diagonale) dans le compartiment, elle manque d'une bonne accroche adaptée
- 2- Le système d'allumage, la batterie doit être allumée avant de pouvoir démarrer le tank, ce n'est pas pratique
- 3- La tourelle et son système de fixation, on a travaillé sur un système de clips de la tourelle dans le servo-moteur mais on a manqué de temps afin de pouvoir le finir et l'adapter
- 4- Les chenilles, même si la version actuelle est fonctionnelle, elles pourraient être encore un peu améliorées
- 5- Le SR04 n'a pas de fixation, il faudrait ajouter des inserts
- 6- Le vis de fixation des roulements à bille ont également besoin d'insert et de plus de matière dans laquelle mettre les inserts
- 7- Le système de fixation de la PCB est un peu rustique et mériterait d'être retravaillé